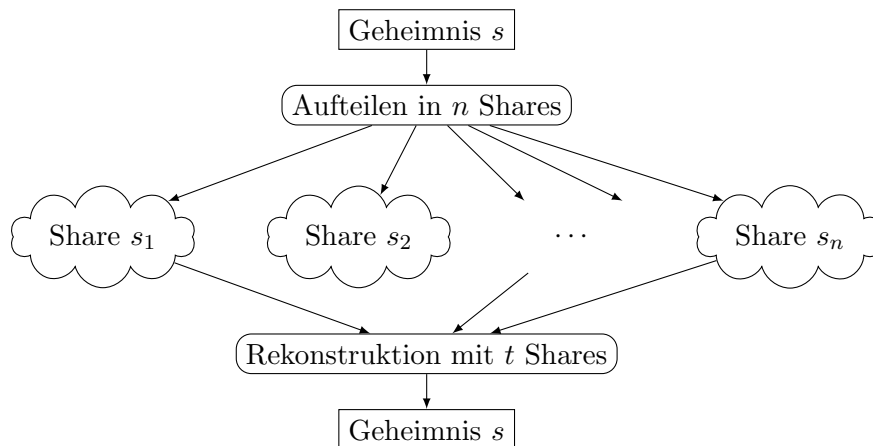


AG IT-Sicherheit

Secret-Sharing

1 Einführung und Orientierung



Unter (t, n) -Secret-Sharing versteht man ein Verfahren, ein Geheimnis (z.B. einen kryptografischen Schlüssel, ein Passwort, eine Safe-Kombination oder auch eine ganze Datei) so in n Teile (*Shares* genannt) aufzuteilen, dass man mindestens t Teile kombinieren muss, um das ursprüngliche Geheimnis wiederherstellen zu können. Falls nur $t - 1$ oder weniger Shares vorhanden sind, soll es informationstheoretisch unmöglich sein, etwas über das Geheimnis zu erfahren: alle möglichen Geheimnisse sind dann immer noch gleich wahrscheinlich.

Es ist sofort einleuchtend, dass Secret-Sharing nur für $n \geq t \geq 2$ sinnvoll ist. Im einfachsten Fall haben wir $t = n$ (dann werden alle Shares zur Rekonstruktion benötigt), aber $t < n$ ist immer dann sinnvoll, wenn die Wiederherstellung trotz des Ausfalls einiger Shares immer noch funktionieren soll.

Eine mögliche Anwendung eines $(2, 4)$ -Secret-Sharing besteht zum Beispiel darin, die Shares einer Datei mit wichtigem Inhalt auf 4 Cloud-Provider verteilt abzuspeichern (etwa Google, Amazon, Microsoft und Dropbox). Dann müssten für einen erfolgreichen Angriff zwei Benutzerkonten kompromittiert werden oder zwei Cloud-Anbieter miteinander kooperieren, während gleichzeitig der Ausfall zweier Anbieter für die Wiederherstellung kein Problem darstellt.

1.1 Einfaches (n, n) -Secret-Sharing

Für diesen Spezialfall existiert ein einfaches Verfahren, das auf binärem XOR basiert. Zur Erinnerung: die XOR-Verknüpfung zweier Bits ist definiert als

$$\begin{array}{c|cc} \oplus & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 0 \end{array},$$

und kann trivial auf ganze Folgen von Bytes verallgemeinert werden. In Java entspricht dies dem Operator `^`, der ganz einfach für alle Elemente eines `byte`-Arrays iteriert werden kann.

Wir werden weiterhin eine Zufallszahlenquelle benötigen. In Java gibt es dafür die Klasse `java.security.SecureRandom`, deren Methode `nextBytes` ein Byte-Array mit neuen Zufallszahlen füllt.

Ein Geheimnis s der Länge l Bytes kann dann wie folgt auf n Shares $s_1, \dots, s_n$ verteilt werden:

1. Für $i = 1, \dots, n-1$: berechne eine neue zufällige Bytesequenz r_i der Länge l und setze $s_i := r_i$.
2. Setze $s_n := s \oplus \bigoplus_{i=1}^{n-1} s_i = s \oplus s_1 \oplus \dots \oplus s_{n-1}$.

Alle n Shares sind jeweils ebenfalls l Bytes lang. Die Rekonstruktion des Geheimnisses s aus den n Shares $s_1, \dots, s_n$ erfolgt analog:

1. Setze $s := \bigoplus_{i=1}^n s_i = s_1 \oplus \dots \oplus s_n$.

Wegen $s_i \oplus s_i = 0$ sieht man die Korrektheit sofort ein (XOR ist sein eigenes Inverses). Sind weniger als n Shares vorhanden, kann das fehlende Share jedes Bit mit Wahrscheinlichkeit $1/2$ kippen – man ist also genauso schlau wie ohne Kenntnis der $n-1$ Shares.

1.2 Shamirs (t, n) -Secret-Sharing

Soll jede Teilmenge von t Shares, $2 \leq t \leq n$, die Rekonstruktion des Geheimnisses ermöglichen, benötigt man ein anderes Verfahren, das 1979 vom Turing-Preisträger Adi Shamir entwickelt wurde. Während das XOR-Verfahren mit Eingaben beliebiger Länge arbeiten kann, erfordert Shamirs Verfahren Geheimnisse s , die sich als Zahl in der Form $0 \leq s < p$ für eine große Primzahl p darstellen lassen. (In der Praxis ist das keine Einschränkung, siehe Abschnitt 1.)

Ein Geheimnis s dieser Form kann dann wie folgt auf n Shares $s_1, \dots, s_n$ verteilt werden, von denen jede Teilmenge von mindestens t Shares zur Rekonstruktion ausreicht:

1. Für $i = 1, \dots, t-1$: berechne eine neue Zufallszahl a_i , $0 \leq a_i < p$.
2. Betrachte das Polynom $f(x) = s + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$ und berechne für $i = 1, \dots, n$ das Share $s_i := (x_i, y_i) = (i, f(i) \bmod p)$.

Das i -te Share s_i besteht also aus dem Punkt mit der x-Koordinate i und der y-Koordinate $f(i) \bmod p$, wobei das Polynom modulo p evaluiert wird.

Zur Rekonstruktion von s benötigen wir den Wert $f(0) \bmod p$, den wir durch Interpolation der vorliegenden Punkte errechnen können: t oder mehr Punkte bestimmen das Polynom f vom Grad $t-1$ eindeutig. Folglich können wir die wohlbekannte Lagrange-Interpolation verwenden, um aus einer beliebigen Teilmenge von wenigstens t Shares das Polynom und damit auch $f(0) \bmod p$ wiederherzustellen. Angenommen, wir verfügen über die k Shares $s_1 = (x_1, y_1), \dots, s_k = (x_k, y_k)$, wobei $t \leq k \leq n$. Dann können wir das Geheimnis s rekonstruieren als

$$s := f(0) \bmod p = \sum_{i=1}^k v_i \bmod p$$

mit

$$v_i = y_i \cdot \prod_{\substack{1 \leq j \leq k, \\ j \neq i}} \frac{-x_j}{x_i - x_j} \bmod p.$$

Bei der Implementierung ist hier zu beachten, dass alle Berechnungen modulo p ausgeführt werden müssen. Hierbei (und bei den großen Zahlen) hilft die Klasse `java.math.BigInteger` enorm, welche wie folgt verwendet werden kann:

```
// Gegeben BigIntegers a, b, c und p,  
// berechne  $d = a / (b-c) = a * 1/(b-c) \text{ modulo } p$   
BigInteger d = a.multiply(b.subtract(c).modInverse(p)).mod(p);
```

Division wird also durch Multiplikation mit dem Inversen erreicht. Am Ende jeder Berechnung muss modulo p reduziert werden.

2 Aufgaben (1. Tag)

Der Aufgabenstellung beiliegend finden Sie ein Programmskelett, welches Sie verwenden können, aber natürlich nicht müssen. Lediglich die geforderte Funktionalität sollte übereinstimmen.

Aufgabe 1 (Einfaches (n, n) -Secret-Sharing mit XOR, 2+2=4 Punkte)

Wir betrachten zunächst das Verfahren aus Abschnitt 1.1.

- a) Implementieren Sie dieses Verfahren (Sharing und Wiederherstellung) für allgemeines n und für Geheimnisse, die in Form von `byte[]`-Arrays beliebiger Länge vorliegen. Testen Sie die Korrektheit Ihrer Implementierung an einigen Beispielen. Die Datei `XorSecretSharing.java` enthält einen Vorschlag für die Schnittstellen für das Sharing und die Wiederherstellung.
- b) Schreiben Sie ein einfaches Programm, welches die folgenden Funktionalitäten bietet:
 - a) Aufteilung einer Datei F mit dem in a) implementierten Secret-Sharing-Verfahren in n Share-Dateien $F-1, F-2, \dots, F-n$.
 - b) Rekonstruktion der Datei F aus den n Einzeldateien.

Aufgabe 2 (Shamirs (t, n) -Secret-Sharing, 4+2=6 Punkte)

Wir implementieren jetzt Shamirs Verfahren aus Abschnitt 1.2. Aus Gründen der Einfachheit lassen wir die Umwandlung von binären Daten oder Dateien in Ganzzahlen und zurück für heute zunächst weg.

- a) Implementieren Sie dieses Verfahren (Sharing und Wiederherstellung) für allgemeines t und n und für Geheimnisse, die bereits in Form von `BigInteger`s vorliegen. Als Primzahl nehmen wir $p = 2^{256} + 297$, so dass wir im Prinzip Geheimnisse bis zu $256/8 = 32$ Byte Länge als `BigInteger` kodieren und verarbeiten könnten. Für die Generierung der zufälligen Koeffizienten a_i ist folgender `BigInteger`-Konstruktor hilfreich:

```
BigInteger ai = new BigInteger(p.bitLength(), rng);
```

wobei `rng` ein `SecureRandom`-Objekt ist. Für die Evaluierung des Polynoms empfiehlt sich das Horner-Schema¹.

¹<https://de.wikipedia.org/wiki/Horner-Schema>

Die Dateien `ShamirSecret.java` und `ShamirSecretSharing.java` enthalten einen Vorschlag für die Schnittstellen für das Sharing und die Wiederherstellung sowie ein paar andere Vorarbeiten (z.B. die Definition von p als `BigInteger`). Beachten Sie auch, dass die x-Koordinaten für die Shares $s_1, s_2, \dots, s_n$ wirklich bei 1 und nicht bei 0 beginnen müssen, da sonst $s_1 = f(0) \bmod p = s$ wäre, was dem Zweck des Secret-Sharings widerspricht.

- b) Demonstrieren Sie die Korrektheit Ihrer Implementierung an einigen Beispielen, in denen Sie `BigIntegers` teilen und rekonstruieren (oder noch besser: schreiben Sie einen vernünftigen JUnit-Test für Ihre Shamir-Secret-Sharing-Klasse).
- c) (Bonuspunkt) In Spezialfällen möchte man bei der Rekonstruktion eines Geheimnisses verschieden große Kombinationen von unterschiedlichen Parteien ermöglichen. Als Beispiel: In einem Unternehmen gibt es n_1 Admins und n_2 Entwickler. Für ein Geheimnis sollen t_1 Admins oder t_2 Entwickler ausreichen, um es zu rekonstruieren. Überlegen Sie sich, wie man ein solches Szenario konzeptionell mit Shamirs Verfahren modellieren kann. Was sind Vor- und Nachteile Ihrer Idee?

AG IT-Sicherheit

Secret-Sharing

1 Aufgaben (2. Tag)

Heute kombinieren wir unsere gestern entwickelte Secret-Sharing-Funktionalität nach dem Shamir-Verfahren mit Datenverschlüsselung: Die zu sharende Datei F wird mit einem frisch erzeugten zufälligen Schlüssel k verschlüsselt, der dann selbst in n Shares $k_1, \dots, k_n$ aufgeteilt wird. Zur Rekonstruktion braucht man dann t der n Schlüssel-Shares sowie die verschlüsselte Datei. Empfehlenswerte Schlüssellängen für k liegen heutzutage bei 128 oder 256 Bits.

Dieser Ansatz hat vor allem den Vorteil, besser für beliebige Dateigrößen zu skalieren. Ansonsten müssten wir für eine große Datei auch eine entsprechend große Primzahl verwenden, was unpraktisch und vor allem extrem ineffizient wäre.

Die korrekte und sichere Verwendung kryptografischer Funktionalität ist nicht trivial. Das Verschlüsselungs-API im JDK etwa ist für Entwickler ohne Spezialkenntnisse voller Fallstricke. Die Fernet-Bibliothek:

<https://github.com/l0s/fernet-java8>

stellt eine relativ unkomplizierte Schnittstelle zum Ver- und Entschlüsseln von Daten bereit, weshalb wir sie hier auch verwenden wollen. Die notwendige jar-Datei findet sich in der Beschreibung der GitHubseite im Abschnitt „Adding this to your project“. Folgt den folgenden verlinkten Texten „The Central Repository“ → „1.5.0/“ → „fernet-java8-1.5.0.jar“, um die Jar-Datei herunterzuladen. Alternativ ist hier der direkte Downloadlink

<https://repo1.maven.org/maven2/com/macasaet/fernet/fernet-java8/1.5.0/fernet-java8-1.5.0.jar>.

Zum Schluss kombinieren wir Shamirs Verfahren mit Error Correcting Codes (ECC). Die Idee von ECC ist, dass Daten mit Redundanz transformiert werden, sodass bei der Rekonstruktion eine gewisse Menge an Fehler die Wiederherstellung nicht verhindert. Typische Szenarien der Fehlerentstehung kann das Senden selbst oder ungewollte Bitflips der Festplatte sein. Eine bekannte Technik für ECC sind die Reed-Solomon-Codes. Die ZXing-Bibliothek

<https://github.com/zxing/zxing>

bietet eine einfache Möglichkeit, ein `int`-Array mittels Reed-Solomon-Codes zu ent- und dekodieren. Der direkte Downloadlink für die Jar-Datei ist

<https://repo1.maven.org/maven2/com/google/zxing/core/3.5.4/core-3.5.4.jar>.

Aufgabe 3 (Dateiverschlüsselung mit der Fernet-Bibliothek, 2 Punkte)

a) Integrieren Sie die Fernet-Bibliothek in Ihre Entwicklungsumgebung und schreiben Sie ein Programm, welches mit Hilfe der Fernet-Bibliothek eine Datei verschlüsseln und wieder entschlüsseln kann. Beachten Sie dabei folgende Punkte:

- Das Programm soll eine Datei `F` zur Datei `F.enc` verschlüsseln, jene dann neu einlesen und zur Datei `F.dec` entschlüsseln.
- Der 256-Bit (32-Byte)-Schlüssel sollte mit einem `SecureRandom`-Objekt erzeugt und als `Key`-Objekt in die Fernet-Bibliothek geladen werden.

- Verwenden Sie bei der Entschlüsselung die vorgegebene Klasse `BytesValidator` als Validator. Fernet unterstützt direkt nur Strings, was für Binärdateien lästig ist, und erlaubt standardmäßig außerdem die Entschlüsselung nur bis zu 60 Sekunden nach der Verschlüsselung (time-to-live). Die `BytesValidator`-Klasse löst diese beiden Probleme.

Demonstrieren Sie die Korrektheit durch einen Vergleich der entschlüsselten Datei mit der Originaldatei (gerne auch manuell, z.B. mit `cmp`, `md5sum`, ...).

Aufgabe 4 (Kombination von Secret-Sharing mit Verschlüsselung, 3+1=4 Punkte)

- a) Erweitern Sie das Programm, indem Sie zusätzlich zur Verschlüsselung der Datei `F` den 32-Byte-Schlüssel `k` mit Hilfe von Shamirs Secret-Sharing in n Shares aufteilen und in den Dateien `F.key.1` bis `F.key.n` abspeichern.

Das Programm soll eine Datei verschlüsseln und den Schlüssel in Share-Dateien aufteilen können; und umgekehrt ebenso die Rekonstruktion des Schlüssels aus einer geeigneten Anzahl Share-Dateien und anschließende Entschlüsselung der Datei unterstützen.

Einige Hinweise:

- Verwenden Sie zur Wandlung des als Byte-Array vorliegenden Schlüssels `k` in einen `BigInteger` und zurück die vorgegebene Hilfsklasse `BigIntegerUtils`.
 - Die vorgegebene Klasse `ShamirShare` bietet Funktionen zur Serialisierung und Deserialisierung von Shares.
 - Bonuspunkt: Berücksichtigen Sie, dass jede Anzahl von Share-Dateien zwischen t und n (inklusive) zur Rekonstruktion verwendet werden kann.
- b) Demonstrieren Sie die Korrektheit Ihres Programms anhand einer Datei, welche Sie verschlüsseln, den Schlüssel mit den Parametern $(t, n) = (2, 4)$ in 4 Shares aufteilen, und anschließend bei der Entschlüsselung aus nur zwei beliebigen der vier Shares rekonstruieren.

Aufgabe 5 (Fehlerkorrektur in den Share-Objekten, 1 + 2 + 1 = 4 Punkte)

In dieser Aufgabe gehen wir das Szenario durch, bei dem Shares in ein ECC zu transformiert, Fehler eingebaut und anschließend wieder rekonstruiert werden. Da die ZXing-Bibliothek mit `int`-Arrays arbeitet, müssen die `ShamirShare`-Objekte zuerst in die benötigte Form gebracht werden.

- a) Zum Serialisieren und Deserialisieren implementieren Sie die beiden Funktionen `toBytes` und `fromBytes` der Klasse `ShamirShare`.
- b) Integrieren Sie die ZXing-Bibliothek in Entwicklerumgebung und implementieren Sie die Funktionen `encode` und `decode` der Klasse `ErrorCorrectingCode`. Benutzen Sie dabei `ReedSolomonEncoder` bzw. `ReedSolomonDecoder`. Benutzen Sie für beide Klassen den Parameter `GenericGF.QR_CODE_FIELD_256`. Für das Kodieren werden zusätzliche Integer (ECC-Symbole) benötigt. Die Anzahl der benötigten Symbole ist das doppelte der erlaubten Fehlermenge. Beachten Sie, dass diese Anzahl bei Kodierung und Dekodierung identisch sein muss.
- c) Schreiben Sie einen Testfall, in welchem alle Shares eines Shamir-Secret-Sharing zuerst ECC-kodiert werden. Verändern Sie danach einige Elemente der `int`-Arrays. Anschließend sollen die fehlerhaften `int`-Arrays dekodiert und wieder zu `ShamirShare`-Objekten

rekonstruiert werden. Zeigen Sie, dass die Felder des ursprünglichen und des rekonstruierten Shares gleich sind.

Zum Schluss: Wer auf den Geschmack gekommen ist und die hier entwickelten Programme selbst für eigene Geheimnisse einsetzen will, sollte bedenken, dass sicherheitsrelevanter Code unbedingt unabhängig und von Experten auf diesem Gebiet begutachtet werden sollte. Also nicht zuviel Enthusiasmus beim Einsatz des Programms für wirklich wichtige Daten!

Für Interessierte: Fernet verwendet sogenannte authentifizierte Verschlüsselung, d.h. jegliche Modifikation der verschlüsselten Dateien kann zuverlässig erkannt werden. Als Algorithmen kommen hier AES-Verschlüsselung im CBC-Modus und HMAC-SHA-256 zum Einsatz, jeweils mit eigenen 128-Bit-Schlüsseln. Mehr zu lesen gibt es hier: <https://github.com/fernet/spec/blob/master/Spec.md>.